

Обработка заняла 13m 11s

Нурали, делаем один веб-симулятор управления городом: выбираешь пять решений, распределяешь бюджет, видишь последствия для районов и получаешь от AI проверенные рекомендации. Не отдельный чат с красивыми ответами, а работающую модель, где каждое число можно объяснить.

Рабочее название: **QALA AI — «Аким на 5 часов»**.

Ниже — план проекта и разделение работы на троих.

1. Что именно мы создаём

Адаптивный сайт, основной интерфейс — под ноутбук. Отдельное мобильное приложение на хакатоне не делаем: сначала доводим один продукт до рабочего состояния.

Пользователь получает город с пятью районами, изучает проблемы, выбирает мероприятия и сравнивает результат с исходным состоянием. В предоставленном датасете есть десять показателей районов и каталог из четырнадцати мероприятий. Все показатели уже направлены одинаково: **больше — лучше**, поэтому, например, транспортный показатель нельзя повторно «переворачивать».

Датасет районов +1

Главная идея продукта:

«Не просто потратить бюджет. Покажи, почему твой план улучшает город и не оставляет слабые районы без помощи».

При этом в интерфейсе прямо пишем: **«Учебная симуляция на синтетических данных»**, а не выдаём результаты за реальные прогнозы развития Астаны.

Сначала важное уточнение правил

Пять решений — не обязательно по одному из каждого направления. В подробном датасете указано: ровно пять мероприятий, максимум два из одного направления. Поэтому достаточно охватить минимум три направления. Его пример прямо содержит две социальные меры и ни одной транспортной. Предлагаю реализовывать подробные правила датасета, а трактовку краткого кейса подтвердить у организаторов.

Датасет районов +1

Основные ограничения нашей системы:

Правило	Как реализуем
Бюджет — 100 условных единиц	Постоянно показываем расходы и остаток; сервер не принимает превышение
Ровно 5 решений	Пять слотов; официальный результат доступен только для полного допустимого набора
Каждое мероприятие максимум один раз	Нельзя повторить его даже в другом районе
Максимум 2 мероприятия одного направления	Проверяем при выборе и повторно на сервере
Районные и городские меры различаются	Для районной обязателен район, для городской его не передаём
Есть несовместимости	Объясняем, какая мера конфликтует с какой и почему
Остаток бюджета не даёт бонус	Не придумываем дополнительные очки за экономию

Это требования датасета, а не наши дополнительные ограничения.

Датасет районов +1

2. Чем проект будет выделяться

Предлагаю три **сильные дополнительные функции**, которые связаны с задачей и не раздувают разработку.

А. «Кого мы оставили позади?»

Обычного общего балла недостаточно. После расчёта показываем:

Как изменился город в среднем → какой район остался самым слабым → где сохранились критические показатели.

В интерфейсе это будет отдельная панель: район, проблемный показатель, значение до и после, причина оставшегося штрафа.

В исходной формуле уже учитываются самый слабый район и показатели ниже 40. Мы не изобретаем другую систему оценки, а делаем эту логику видимой и понятной.

Датасет районов

Польза: можно увидеть, что общий рост не означает равномерного улучшения жизни.

Б. «Улучши мой план одной заменой»

Пользователь нажимает кнопку, и система ищет улучшение, **не переписывая весь его сценарий**.

Механика: перебираем допустимые замены одного мероприятия или изменение района его применения, проверяем ограничения, рассчитываем результаты и показываем лучший найденный вариант.

На экране:

«Предлагаем заменить эту меру на другую. Вот изменение стоимости, городского балла и состояния слабейшего района».

Важно: варианты проверяет расчётный модуль, а AI объясняет их. AI не придумывает стоимость и не обещает эффект, которого нет в модели.

Называем результат **«лучший найденный вариант с одной заменой»**, а не «идеальный план города». Если улучшения нет — честно сообщаем.

В. «Защити своё решение»

После симуляции AI формирует короткое объяснение для защиты:

Почему выбраны эти меры. Какой компромисс принят. Какие проблемы решены. Какие остались. Почему предложенная замена лучше или хуже.

Рядом располагаются подтверждающие цифры из расчётного модуля.

Это не просто длинный AI-текст: пользователь видит связку **утверждение → показатель → конкретное решение**. Такой подход соответствует указанной в датасете роли LLM: объяснять рассчитанные результаты, сравнивать наборы и советовать, но не считать и не придумывать числа.

Датасет районов

Что оставляем на дополнительное время

Неожиданные события, режим вопросов «от жителей», экспорт презентации и общий рейтинг команд — второй приоритет. События должны жить в отдельном экспериментальном режиме: **не менять незаметно официальные условия и Score**. В кейсе такие возможности перечислены как опциональные.

HackAlem AI_ «Аким на 5 часов» ...

3. Как будет выглядеть сайт

Не делаем десять разрозненных страниц. Основной продукт — **одна рабочая панель и экран результатов**.

Экран 1. Старт

Название, краткая задача, бюджет, количество решений и кнопка **«Начать управление»**.

Рядом — понятные правила и отметка о синтетических данных. Без обязательной регистрации.

Экран 2. Панель управления городом

Область	Содержание
Верхняя полоса	Бюджет, остаток, количество решений: например, 3 из 5
Левая часть	Пять районов; при выборе — десять показателей и основные проблемы
Центральная часть	Каталог мероприятий с фильтрами по направлениям
Правая часть	Выбранные решения, их районы, стоимость и предупреждения
Нижняя часть	Кнопка расчёта и объяснение, чего не хватает для допустимого сценария

В карточке мероприятия показываем не только название и цену, но и **районный/ городской охват, задержку эффекта, положительные и отрицательные изменения**. Датасет предусматривает восьмиквартальный горизонт и эффект с учётом задержки; среди мер есть и отрицательные побочные эффекты.

Датасет районов +1

Для районов достаточно красивой **схематичной интерактивной SVG-карты** с подписью «схема». Географически точные границы и внешние картографические сервисы пока не нужны.

Пока выбрано меньше пяти решений, показываем расходы и предварительные изменения показателей, **но не выдаём официальный итоговый Score**.

Экран 3. Результат

Главный блок:

Score до → Score после → изменение.

Ниже — показатели районов «до/после», критические проблемы, использованные синергии и AI-объяснение.

Действия: «Изменить решения», «Улучшить одной заменой», «Сохранить сценарий».

Экран 4. Сравнение

Два сохранённых сценария рядом: расходы, Score, слабейший район, число критических показателей и различия в решениях.

Для первой версии сохраняем сценарии в браузере. **Не обещаем общий межкомандный рейтинг без серверного хранения.**

4. Как должна работать математика и AI

Это принципиальное разделение:

Расчётный модуль отвечает за истину в рамках модели. AI отвечает за объяснение.

Расчётный модуль

Реализует исходные показатели, веса, стоимость, задержки, синергии и несовместимости **строго из датасета.**

Итоговая формула:

$$Score = 0.7 \times D_{avg} + 0.3 \times D_{min} - N_{crit}$$

Здесь D_{avg} — средневзвешенная по населению оценка районов, D_{min} — оценка слабейшего района, N_{crit} — количество пар «район × показатель» со значением **строго меньше 40**.

Датасет районов

В реализации фиксируем важные детали: сначала складываем эффекты и синергии, затем ограничиваем показатели диапазоном 0–100; синергии не масштабируем задержкой; промежуточные значения не округляем; порядок выбора мероприятий не должен менять результат.

Датасет районов +2

Контрольные примеры для тестов: я перепроверил арифметику по вашим исходным данным. База получается **52,55768**, то есть **52,56**. Пример из файла — М7 в Нуре, М8 в Нуре, М10 в Нуре, М12 по городу и М5 в Сарыарке — стоит **95** и даёт **56,54307**, то есть **56,54**. Эти значения закрепляем автоматическими тестами.

Датасет районов

AI-часть

Сервер передаёт модели уже рассчитанные данные: решения, стоимость, изменения показателей, штрафы, синергии и проверенные альтернативы.

AI возвращает структурированное объяснение: сильные стороны, риски, компромиссы и рекомендации. Числовые карточки интерфейса всегда заполняются из расчётного модуля, не из AI-текста.

При сбое AI сайт продолжает считать и показывает **«AI-анализ временно недоступен»**. Можно вывести шаблонное объяснение, но нельзя выдавать его за работающий AI.

Отдельный организационный момент: ваши подписки ChatGPT Pro используются для разработки. API для AI внутри сайта имеет отдельный биллинг; подписка ChatGPT его не оплачивает. Это нужно проверить в самом начале, а не перед защитой.  OpenAI Help C...

5. Техническая архитектура

Предлагаю следующий стек как рабочее решение для команды:

Часть	Выбор
Интерфейс	React + TypeScript + Vite
Сервер	Node.js + TypeScript + Fastify
Математика	Отдельный TypeScript-пакет без зависимости от интерфейса и AI
Общие форматы данных	Отдельный пакет <code>contracts</code>
Исходные данные	Версионизуемые JSON-файлы в репозитории
Сохранение сценариев	В браузере для первой версии
Организация проекта	Один репозиторий с npm workspaces

Не добавляем базу данных, авторизацию, мобильное приложение и несколько микросервисов без необходимости.

Связь компонентов:

Пользователь



Веб-интерфейс



Сервер API

- └ Расчётный модуль → проверка, показатели, Score, альтернативы
- └ AI → объяснение готовых результатов

В окончательном результате сервер повторно проверяет решения. Присланные браузером цены, веса или готовый Score не принимаются за истину.

6. Как делим работу между тремя участниками

Делим не по страницам, а по независимым слоям. Тогда каждый может разрабатывать свою часть и проверять её отдельно.

Участник 1 — интерфейс

Его задача: всё, что пользователь видит и нажимает.

Он делает панель районов, каталог, пять слотов решений, бюджет, ошибки, экран результатов, сравнение сценариев и отображение AI-ответа.

Его зона:

```
apps/web/  
docs/frontend.md
```

Критерий готовности: пользователь проходит весь путь от старта до результата; интерфейс работает с согласованными тестовыми ответами API и затем подключается к настоящему серверу.

Не делает: собственную формулу Score, серверные маршруты и обращения к AI напрямую из браузера.

Участник 2 — данные, математическая модель и подбор альтернатив

Его задача: сделать правильный и проверяемый «двигатель города».

Он переносит датасет в JSON, реализует валидатор, формулу, задержки, синергии, несовместимости, изменения показателей и поиск улучшения одной заменой.

Его зона:

```
packages/engine/  
  data/  
  src/  
  tests/  
docs/model.md
```

Критерий готовности: модуль получает решения и возвращает либо причины недопустимости, либо полный расчёт. Контрольные примеры совпадают, тесты проходят.

Не делает: интерфейс, HTTP-сервер и AI-промпы.

Участник 3 — сервер, AI и сборка проекта

Его задача: соединить компоненты и обеспечить запуск всего продукта.

Он создаёт API, подключает расчётный модуль, реализует AI-анализ, обработку ошибок, конфигурацию окружения, общие контракты, интеграционные проверки и развёртывание.

Его зона:

```
apps/api/  
packages/contracts/  
tests/e2e/  
docs/integration.md  
.github/  
README.md  
AGENTS.md
```

Он же — **координатор интеграции**, а не человек, который переписывает работу остальных.

Критерий готовности: чистая копия репозитория запускается по README; интерфейс получает настоящий расчёт и настоящий AI-ответ.

Общие файлы — отдельное правило

Все `package.json`, общий lock-файл, конфигурации сборки и общие типы **меняет только участник 3 по согласованию**. Даже когда такой файл находится внутри папки другого участника.

Именно здесь легко получить конфликт, несмотря на разделение основных папок.

7. Что согласовываем до параллельной разработки

Нельзя просто раздать роли и попросить каждый GPT «сделать свою часть». Сначала нужен единый технический договор.

Участник 3 создаёт каркас репозитория, а команда согласовывает четыре API-запроса:

Запрос	Назначение
GET /api/bootstrap	Районы, мероприятия, правила, бюджет, исходный результат и версия модели
POST /api/simulate	Проверка пяти решений и расчёт
POST /api/recommend	Поиск проверенных улучшений одной заменой
POST /api/explain	AI-объяснение сценария, заново рассчитанного сервером

Фиксируем единые идентификаторы районов и мероприятий, названия полей, форматы ошибок и результатов.

Например, одно решение содержит **ID мероприятия** и, когда требуется, **ID района**. Невалидный набор возвращает причины ошибок и `score: null`, а не выдуманный нулевой балл.

В `packages/contracts` также кладём согласованные тестовые ответы. Благодаря этому первый участник делает интерфейс, **не ожидая готовности сервера**, а второй — расчёты, не ожидая интерфейса.

8. Как работать в общем GitHub без постоянных конфликтов

Полностью исключить ошибки обещать нельзя. Но можно убрать большую часть причин конфликтов.

У каждого — своя локальная копия всего репозитория, но изменения только в своей зоне. Для отдельных задач создаются ветки, например:

```
feat/ui-scenario-builder
feat/engine-validation
feat/api-ai-analysis
```

Рабочий цикл: обновить `main` → создать ветку → реализовать небольшой блок → проверить → открыть Pull Request → после проверки объединить. Это соответствует веточному процессу GitHub Flow.  GitHub D...

Договариваемся о следующих правилах:

1. В `main` **напрямую не пишем**. Объединение выполняет координатор после проверок.

2. **Чужие файлы читаем, но не меняем.** Нужно изменение передаём владельцу.
3. **Не переписываем архитектуру посреди работы.** Изменение общего формата обсуждается до реализации.
4. **Не добавляем зависимости самостоятельно.** Запрос координатору — он обновляет зависимости и lock-файл.
5. **Объединяем небольшие законченные части,** а не три огромных проекта в последний момент.

После каждого объединения проверяем **сборку всего проекта**, а не только папки автора. Отсутствие Git-конфликта ещё не означает, что компоненты совместимы.

Что должны знать все три GPT-чата

В репозитории создаём `AGENTS.md` с общей задачей, владельцами папок, запретами и командами проверки. Каждый участник даёт своему чату одинаковое общее описание и свою роль.

Основная инструкция:

Ты работаешь над частью общего проекта QALA AI. Сначала прочитай AGENTS.md и общие контракты. Изменяй только файлы своей зоны. Не меняй форматы API, зависимости и чужие модули без согласования. Реализуй один законченный блок за раз. После изменений укажи изменённые файлы, выполненные проверки и оставшиеся зависимости от других участников. Не заявляй, что тесты прошли, если они не запускались.

Не рассчитываем на общую память трёх чатов. Все важные договорённости храним в репозитории.

9. Порядок разработки

Этап	Что делаем	Условие перехода дальше
1. Общий каркас	Папки, зависимости, контракты, тестовые ответы, правила работы	Все трое запускают один и тот же стартовый проект
2. Базовые части параллельно	Первый — интерфейс; второй — валидатор и Score; третий — API и подключение AI	Каждый модуль проверяется отдельно
3. Первый сквозной сценарий	Выбор пяти мер → API → расчёт → результат на экране	Контрольный пример работает через сайт

Этап	Что делаем	Условие перехода дальше
4. Отличительные функции	Панель слабых районов, улучшение одной заменой, сравнение и AI-защита	Рекомендации действительно пересчитываются
5. Финализация	Ошибки, тесты, README, запуск на чистой копии, репетиция	Продукт можно воспроизвести без устных подсказок

Первый сквозной запуск делаем раньше красоты и дополнительных режимов. Уже на третьем этапе у вас должен быть демонстрируемый продукт.

10. Что обязательно проверить перед сдачей

Проверяем не только счастливый путь.

Проверка	Ожидаемое поведение
Стоимость больше 100	Отказ с понятной причиной
Выбрано 4 или 6 решений	Официальный Score не рассчитывается
Повтор мероприятия	Отказ, даже при другом районе
Три меры одного направления	Отказ
Несовместимые мероприятия	Отказ с объяснением конфликта
Значение показателя ровно 40	Критического штрафа нет
Перестановка тех же решений	Результат не меняется
Одинаковый запрос повторно	Те же расчётные числа
Ошибка AI или отсутствие ключа	Расчёты работают, отсутствие AI обозначено
Свежая копия репозитория	Запуск проходит по README

В README включаем установку, переменные окружения, запуск, тесты, архитектуру, формулу, источники данных, контрольный сценарий и ограничения модели.

Документация здесь не второстепенна: за README и воспроизводимость дают **25 баллов**, столько же, сколько за техническую реализацию.

11. Что показываем жюри

Демонстрация должна рассказывать одну понятную историю.

Открываем город и показываем исходные проблемы. Собираем допустимый план, по ходу демонстрируем защиту от нарушения бюджета или несовместимости. Получаем результат и объясняем, почему вырос общий балл, но отдельные проблемы остались.

Затем нажимаем **«Улучшить одной заменой»**, применяем проверенную рекомендацию и показываем сравнение двух сценариев. Завершаем краткой AI-защитой решения и демонстрацией воспроизводимого запуска и тестов.

Итоговое разделение: первый участник делает удобный город на экране, второй — правильную модель города, третий — сервер, AI и соединение всего в работающий продукт. Именно эти три части и стоит параллельно вайбкодить — с общими контрактами, но без совместного редактирования одних и тех же файлов.

Источники